

2. Extração de Features — Flow Table Controller

2.1 Objetivo da entrega

O Flow Table Controller é responsável por coordenar o processamento dos metadados extraídos de cada pacote, consultar a Flow Table e executar as operações necessárias para manter as estatísticas dos fluxos de rede.

O processamento é dividido em três possibilidades:

- **MISS:** a posição selecionada está livre, sendo necessário criar uma nova entrada de fluxo.
- **HIT:** a posição está ocupada por um fluxo com a mesma chave, sendo necessário atualizar suas estatísticas.
- **COLLISION:** a posição está ocupada por outro fluxo, cuja chave é diferente da chave recebida.

A lógica de atualização das features será realizada diretamente pelo Controller, sem a utilização de um Feature Update Unit separado nesta versão.

O acesso físico à memória será realizado por meio do Data Mover, responsável pelas operações de leitura e escrita. O Controller determina os endereços, interpreta os dados lidos e calcula os valores que devem ser armazenados.

Além do processamento dos pacotes, o Controller também coordena a finalização dos fluxos após o término da leitura do PCAP. Nessa etapa, percorre a Flow Table e encaminha as entradas válidas ao Flow Finalizer.

2.2 Entradas e informações utilizadas pelo Controller

O Controller recebe os metadados produzidos pelas etapas anteriores da arquitetura. Essas informações devem estar disponíveis simultaneamente, garantindo que todos os campos correspondam ao mesmo pacote.

Sinal	Largura	Função
clk	1 bit	Sinal de clock do módulo.
rst_n	1 bit	Reset ativo em nível baixo.
in_valid	1 bit	Indica que os metadados do pacote são válidos.
in_ready	1 bit	Indica que o Controller está disponível para receber um novo pacote.
in_flow_key	104 bits	Identificador do fluxo formado pela tupla de cinco campos.
in_hash_index	16 bits	Índice da entrada da Flow Table, calculado previamente pela Hash Unit.

Sinal	Largura	Função
in_packet_length	16 bits	Comprimento do pacote utilizado na atualização de byte_count.
in_timestamp	80 bits	Timestamp associado ao pacote.
pcap_done	1 bit	Indica que o processamento do PCAP terminou.
ctrl_restart	1 bit	Autoriza o retorno do Controller ao estado inicial após a finalização.

A chave do fluxo possui 104 bits e é composta por:

src_ip + dst_ip + src_port + dst_port + protocol

Ou seja:

$32 + 32 + 16 + 16 + 8 = 104$ bits.

O projeto considera inicialmente o modelo uniflow, no qual a inversão dos endereços e das portas de origem e destino resulta em uma chave diferente.

O índice in_hash_index possui 16 bits e permite selecionar uma entre 65.536 posições da Flow Table. Esse índice já chega calculado ao Controller, que fica responsável por convertê-lo em endereço de memória.

A largura de 80 bits do timestamp foi definida como referência para o projeto. Entretanto, o formato exato, a resolução e a divisão entre parte inteira e fracionária ainda precisam ser alinhados com o TSU.

2.3 Entregável 1 — Fluxograma do Controller

O fluxo de processamento do Controller segue a sequência:

Recepção → Consulta à memória → Verificação → MISS/HIT/COLLISION → Escrita, quando necessária → Próximo pacote.

Após o término do PCAP, o Controller deixa de receber pacotes e inicia a varredura da Flow Table para enviar os registros válidos ao Flow Finalizer.

2.3.1 Início do processamento

O Controller permanece no estado IDLE enquanto não houver um pacote válido.

A transferência de um pacote acontece quando:

`in_valid && in_ready = 1`

Nesse momento, o Controller captura os metadados necessários:

- flow_key;
- hash_index;
- packet_length;
- timestamp.

Essas informações são armazenadas internamente para que permaneçam estáveis durante todo o processamento do pacote.

Após aceitar um pacote, o Controller desativa `in_ready` até concluir a operação atual. Essa estratégia evita que novos metadados sobrescrevam os dados do pacote que está sendo processado.

2.3.2 Consulta à Flow Table — FETCH

No estado FETCH, o Controller calcula o endereço-base da entrada utilizando o índice recebido:

$\text{base_address} = \text{hash_index} \times 32$

Como cada entrada lógica ocupa 32 palavras de 16 bits, multiplicar o índice por 32 equivale a deslocá-lo cinco posições à esquerda.

O Controller solicita ao Data Mover a leitura dos primeiros oito words da entrada. Esses words contêm o campo `valid` e a chave do fluxo, permitindo verificar se a posição está livre ou se pertence ao fluxo recebido.

A solicitação de leitura é feita por meio de `rd_en`. O Controller aguarda `rd_valid` para identificar que os dados solicitados estão disponíveis.

Após receber os dados, o Controller compara o bit `valid` e a chave armazenada com a chave recebida.

2.3.3 Caminho de MISS

O MISS ocorre quando:

`valid = 0`

Isso indica que a posição selecionada está livre e pode ser utilizada para registrar um novo fluxo.

O Controller prepara uma entrada com os seguintes valores:

Campo	Valor inicial
<code>valid</code>	1
<code>flow_key</code>	<code>in_flow_key</code>
<code>packet_count</code>	1
<code>byte_count</code>	<code>in_packet_length</code>
<code>first_timestamp</code>	<code>in_timestamp</code>
<code>last_timestamp</code>	<code>in_timestamp</code>

O primeiro pacote do fluxo é contabilizado imediatamente. Por isso, `packet_count` começa em 1 e `byte_count` recebe o tamanho do pacote que originou a entrada.

Os timestamps inicial e final recebem o mesmo valor, pois, naquele momento, o fluxo possui apenas um pacote registrado.

O Controller solicita ao Data Mover a escrita da nova entrada na posição correspondente. A operação é considerada concluída quando o Data Mover sinaliza `wr_done`.

Depois da escrita, o Controller retorna ao estado de espera para receber o próximo pacote.

2.3.4 Caminho de HIT

O HIT ocorre quando:

`valid = 1`

e

`stored_flow_key = in_flow_key`

Nesse caso, a posição consultada contém o mesmo fluxo do pacote recebido.

O Controller lê os campos necessários da entrada e calcula os novos valores das estatísticas:

`packet_count = packet_count + 1`

`byte_count = byte_count + in_packet_length`

`first_timestamp = first_timestamp`

`last_timestamp = in_timestamp`

O campo `first_timestamp` permanece inalterado, pois representa o instante de chegada do primeiro pacote do fluxo. Já `last_timestamp` é atualizado com o timestamp do pacote mais recente.

Após calcular os novos valores, o Controller solicita ao Data Mover a escrita dos campos atualizados. A entrada somente é considerada atualizada após a confirmação da operação de escrita.

Concluída a atualização, o Controller retorna ao estado IDLE.

2.3.5 Caminho de COLLISION

A colisão ocorre quando:

`valid = 1`

e

`stored_flow_key != in_flow_key`

Nesse caso, o índice calculado pelo hash aponta para uma posição ocupada por outro fluxo.

Para a primeira versão do projeto, será adotado um tratamento simplificado:

- o Controller identifica a colisão;
- a entrada existente é preservada;
- o pacote recebido não é inserido na Flow Table;
- a ocorrência é sinalizada por um evento de colisão;
- o Controller retorna ao estado de espera.

Esse tratamento evita a sobrescrita de dados de outro fluxo. A implementação de estratégias de resolução de colisões poderá ser considerada em versões futuras.

2.3.6 Caminho de finalização

Quando pcap_done é ativado e o Controller está disponível, o processamento de novos pacotes é encerrado e a etapa de finalização é iniciada.

O Controller percorre sequencialmente as 65.536 posições da Flow Table.

Para cada posição:

- se valid = 0, a entrada é ignorada e a varredura avança;
- se valid = 1, os dados do fluxo são enviados ao Flow Finalizer.

O Finalizer recebe os campos necessários para gerar as features finais. Entre suas operações está o cálculo da duração do fluxo:

$duration = last_timestamp - first_timestamp$

O Controller aguarda a aceitação e a conclusão do processamento de cada registro antes de avançar para o próximo.

Após a última posição ser verificada e o último registro válido ter sido processado, o Controller ativa all_flows_done. Esse sinal permanece ativo no estado DONE até que o sistema autorize o reinício por meio de ctrl_restart.

2.4 Entregável 2 — Primeira versão da FSM

A máquina de estados finitos (FSM) controla a sequência das operações do Controller. Cada estado representa uma etapa do processamento, e as transições dependem dos sinais de entrada e das confirmações dos módulos externos.

A proposta considera os seguintes estados:

Estado	Função
IDLE	Aguarda a chegada de um novo pacote ou o sinal de término do PCAP.
FETCH	Solicita a leitura dos dados da entrada da Flow Table.
CHECK	Analisa os dados retornados e classifica o resultado como MISS, HIT ou COLLISION.
MISS	Prepara os dados de uma nova entrada.
INSERT	Solicita e aguarda a escrita de uma nova entrada na memória.
HIT	Prepara os cálculos de atualização das estatísticas do fluxo.
UPDATE	Solicita e aguarda a escrita dos valores atualizados.
COLLISION	Registra e sinaliza uma colisão sem alterar a entrada existente.

Estado	Função
FINALIZE_SCAN	Percorre as posições da Flow Table durante a finalização.
FINALIZE_SEND	Apresenta uma entrada válida ao Flow Finalizer.
FINALIZE_WAIT	Aguarda a conclusão do processamento do registro pelo Finalizer.
DONE	Indica que a varredura e a finalização de todos os registros foram concluídas.

2.4.1 Transições da FSM

Estado atual	Condição	Próximo estado	Operação
IDLE	in_valid && in_ready	FETCH	Captura os metadados do pacote.
IDLE	pcap_done	FINALIZE_SCAN	Inicia a varredura da Flow Table.
FETCH	rd_valid	CHECK	Recebe os dados lidos da memória.
CHECK	valid = 0	MISS	Identifica uma posição livre.
CHECK	valid = 1 e chave igual	HIT	Identifica um fluxo existente.
CHECK	valid = 1 e chave diferente	COLLISION	Identifica uma colisão.
MISS	Próximo ciclo	INSERT	Prepara a nova entrada.
INSERT	wr_done	IDLE	Conclui a inserção.
HIT	Próximo ciclo	UPDATE	Calcula os novos valores das estatísticas.
UPDATE	wr_done	IDLE	Conclui a atualização.
COLLISION	Tratamento concluído	IDLE	Sinaliza a colisão e descarta o pacote.
FINALIZE_SCAN	Entrada inválida e não é a última	FINALIZE_SCAN	Avança para a próxima posição.
FINALIZE_SCAN	Entrada válida	FINALIZE_SEND	Prepara o envio ao Finalizer.

Estado atual	Condição	Próximo estado	Operação
FINALIZE_SEND	finalizer_valid && finalizer_ready	FINALIZE_WAIT	Transfere o registro ao Finalizer.
FINALIZE_WAIT	finalizer_done e ainda há posições	FINALIZE_SCAN	Avança a varredura.
FINALIZE_SCAN	Última posição processada e sem registro pendente	DONE	Encerra a varredura.
FINALIZE_WAIT	finalizer_done para o último registro	DONE	Conclui a finalização.
DONE	ctrl_restart	IDLE	Autoriza o reinício do Controller.

A condição de conclusão deve considerar tanto o término da varredura quanto o processamento do último registro válido. Assim, o Controller não deve ativar all_flows_done apenas por alcançar a última posição da memória se ainda houver um registro pendente no Finalizer.

2.5 Entregável 3 — Sequência das operações

A sequência geral permite compreender como o Controller coordena os módulos durante o processamento de um pacote.

Etapa 1 — Recepção

O Controller aguarda a condição:

`in_valid && in_ready = 1`

Quando a transferência acontece, os metadados do pacote são capturados e armazenados internamente.

Etapa 2 — Cálculo do endereço

O Controller utiliza o `in_hash_index`, recebido da Hash Unit, para calcular o endereço-base da entrada:

`base_address = hash_index × 32`

O endereço-base identifica o início do registro lógico na memória.

Etapa 3 — Leitura da entrada

O Controller solicita a leitura da entrada ao Data Mover e aguarda `rd_valid`.

Os primeiros oito words da entrada são suficientes para verificar o bit valid e comparar a chave armazenada com a chave recebida.

Etapa 4 — Classificação

Após a leitura, o Controller verifica:

- o valor do bit valid;
- a chave armazenada;
- a chave recebida.

A classificação é realizada da seguinte forma:

valid = 0

→ MISS

valid = 1 e stored_flow_key = in_flow_key

→ HIT

valid = 1 e stored_flow_key != in_flow_key

→ COLLISION

Etapas 5A — MISS

Para um MISS, o Controller inicializa uma nova entrada:

valid = 1

flow_key = in_flow_key

packet_count = 1

byte_count = in_packet_length

first_timestamp = in_timestamp

last_timestamp = in_timestamp

Em seguida, solicita a escrita da entrada ao Data Mover e aguarda wr_done.

Etapas 5B — HIT

Para um HIT, o Controller lê os campos estatísticos da entrada e calcula:

packet_count = packet_count + 1

byte_count = byte_count + in_packet_length

last_timestamp = in_timestamp

O campo first_timestamp permanece inalterado.

Após os cálculos, os valores atualizados são escritos novamente na Flow Table.

Etapas 5C — COLLISION

Para uma colisão, a entrada existente é preservada. O Controller sinaliza o evento e não armazena o pacote recebido.

Etapa 6 — Próximo pacote

Após a conclusão da leitura e da operação correspondente, o Controller retorna ao estado IDLE e libera `in_ready`.

Essa organização serializa o processamento dos pacotes. Dessa forma, um novo pacote não inicia uma operação antes que a anterior tenha sido concluída.

Etapa 7 — Finalização

Quando o PCAP termina, o Controller inicia a varredura da Flow Table.

As entradas inválidas são ignoradas. As entradas válidas são encaminhadas ao Finalizer, que calcula as features finais. O Controller aguarda a conclusão de cada registro e, ao final, sinaliza `all_flows_done`.

2.6 Controle de valid/ready e backpressure

A interface de entrada utiliza o protocolo valid/ready, no qual uma transferência ocorre quando os dois sinais estão ativos simultaneamente:

`transfer = in_valid && in_ready`

O produtor mantém `in_valid` ativo e os metadados estáveis até que o Controller aceite o pacote.

O Controller utiliza `in_ready` para indicar quando pode receber novos dados. Durante operações de leitura, classificação, escrita e finalização, esse sinal pode permanecer em nível baixo.

Essa abordagem evita que dois pacotes sejam processados simultaneamente sobre a mesma entrada sem um mecanismo de controle de dependências.

Por exemplo, se dois pacotes do mesmo fluxo fossem processados simultaneamente, ambos poderiam ler o mesmo valor de `packet_count` e calcular o mesmo resultado incrementado. Uma das atualizações poderia ser perdida. A serialização evita esse tipo de conflito na primeira versão.

A interface com o Data Mover também utiliza sinais de solicitação e confirmação:

- `rd_en`: solicita uma leitura;
- `rd_valid`: indica que os dados lidos estão disponíveis;
- `wr_en`: solicita uma escrita;
- `wr_done`: indica que a escrita foi concluída.

O Controller deve aguardar as confirmações correspondentes antes de avançar para a próxima operação. O protocolo detalhado de cada transação deverá respeitar a implementação efetiva do Data Mover.

2.7 Entregável 4 — Dependências com os demais módulos

O Controller depende da definição das interfaces dos módulos anteriores e posteriores da arquitetura. O alinhamento dessas interfaces é necessário para garantir que os dados sejam transferidos corretamente e que as responsabilidades não sejam duplicadas.

2.7.1 Dependência com Metadata Extraction e Hash Unit

O Controller recebe os seguintes metadados:

- flow_key;
- hash_index;
- packet_length;
- timestamp;
- valid.

A Hash Unit é responsável por calcular o índice de acesso. O Controller recebe esse índice pronto e o utiliza para calcular o endereço da entrada na memória.

É necessário garantir que todos os metadados estejam disponíveis simultaneamente e correspondam ao mesmo pacote. Também devem ser definidos o momento em que o timestamp é gerado e sua representação exata.

2.7.2 Dependência com o Data Mover e a Flow Table

O Data Mover executa as operações de leitura e escrita solicitadas pelo Controller. A Flow Table corresponde à organização lógica dos registros mantidos na memória.

A interface proposta para o acesso à memória é:

Sinal	Direção	Função
rd_en	Controller → Data Mover	Solicita uma leitura.
rd_addr	Controller → Data Mover	Endereço da leitura.
rd_valid	Data Mover → Controller	Indica que os dados lidos estão disponíveis.
rd_data	Data Mover → Controller	Dados retornados pela leitura.
wr_en	Controller → Data Mover	Solicita uma escrita.
wr_addr	Controller → Data Mover	Endereço da escrita.
wr_data	Controller → Data Mover	Dados que serão escritos.
wr_done	Data Mover → Controller	Indica que a escrita foi concluída.

O Controller calcula os endereços e prepara os dados de escrita. O Data Mover realiza as operações físicas na memória.

A interface definitiva deve considerar a latência da memória e a forma como o Data Mover recebe e retorna os dados. A organização em words de 16 bits também precisa ser respeitada durante a leitura e a escrita dos registros de 512 bits.

2.7.3 Dependência com o Flow Finalizer

Durante a finalização, o Controller fornece ao Finalizer os campos de cada entrada válida.

A interface de transferência utiliza:

- finalizer_valid: indica que há um registro disponível;
- finalizer_ready: indica que o Finalizer pode aceitar o registro;
- finalizer_done: indica que o processamento do registro foi concluído.

A transferência acontece quando:

finalizer_valid && finalizer_ready

O registro enviado contém cinco campos separados, totalizando 360 bits:

Campo	Largura
flow_key	104 bits
packet_count	32 bits
byte_count	64 bits
first_timestamp	80 bits
last_timestamp	80 bits
Total	360 bits

O Finalizer utiliza os timestamps para calcular a duração do fluxo e gerar o vetor final de features.

O sinal finalizer_done deve representar a conclusão do processamento do registro recebido. A definição exata do momento em que o vetor final estará disponível deve ser alinhada com a implementação do Finalizer.

2.8 Pontos que precisam ser definidos

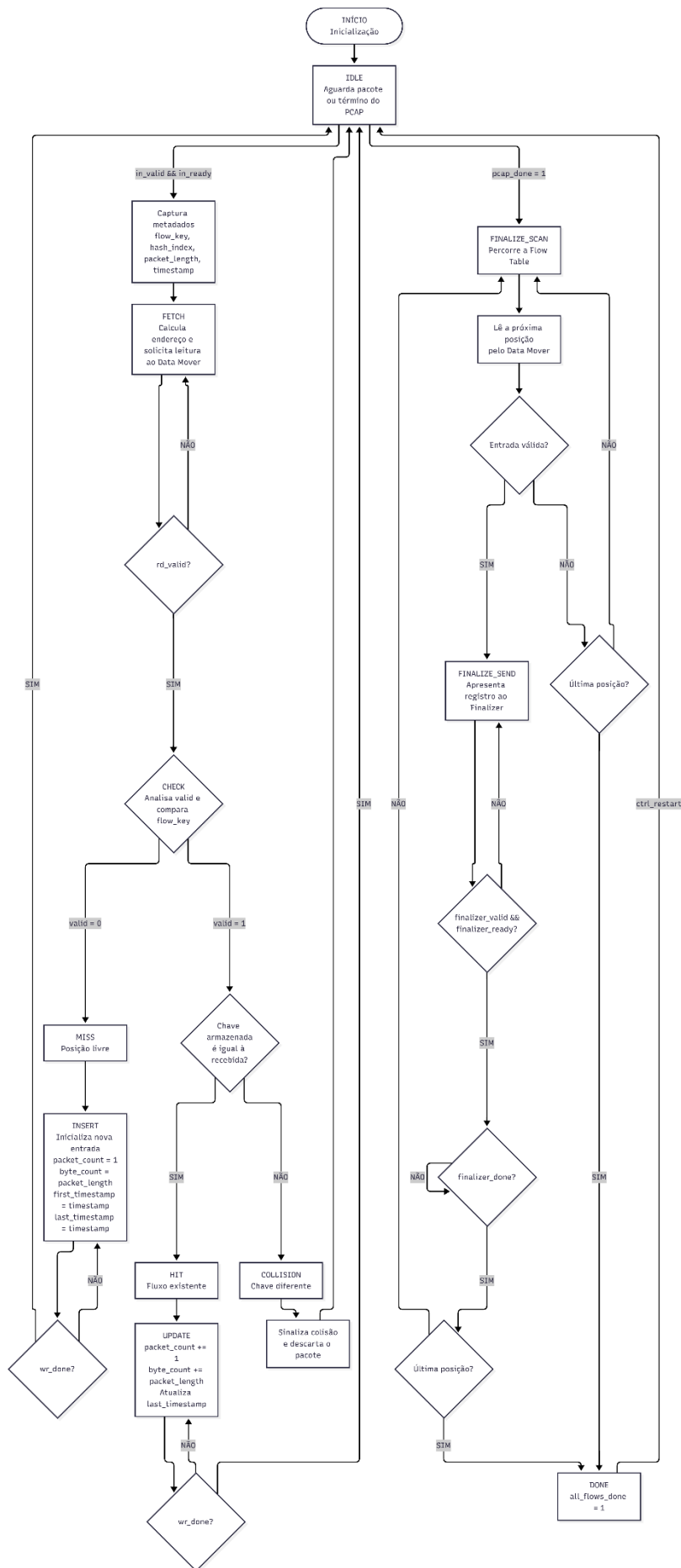
Embora a sequência funcional do Controller esteja estabelecida, algumas características ainda dependem da implementação dos demais módulos.

Questão	Dependência
Qual será a latência efetiva das operações de leitura e escrita?	Data Mover / memória
Como o Data Mover organiza a transferência de words de 16 bits?	Data Mover
Qual será a representação exata do timestamp de 80 bits?	TSU / Metadata Extraction
Como será gerado o timestamp no ambiente de verificação?	Verificação

Questão	Dependência
Como será sinalizada e contabilizada a ocorrência de colisões?	Controller / Verificação
Qual será o formato definitivo do vetor de features produzido pelo Finalizer?	Finalizer / ML
O que exatamente finalizer_done garante: processamento concluído ou vetor final disponível?	Finalizer
Como será realizado o reinício do sistema após all_flows_done?	Integração do sistema
Como serão tratados estouros de packet_count e byte_count?	Controller / especificação das features
Haverá futuramente tratamento de encerramento de fluxo por FIN, RST ou timeout?	Escopo futuro

Esses pontos não impedem a definição da sequência principal da FSM, mas precisam ser alinhados antes do congelamento da interface RTL.

2.9 Fluxograma



Observação: o fluxograma é uma representação funcional. Na implementação RTL, a leitura durante a varredura, o controle do índice e o tratamento da última posição devem ser organizados de modo que nenhuma entrada válida seja ignorada e que o Controller aguarde o Finalizer quando houver um registro pendente.

2.10 Resultado esperado da entrega

Ao final desta etapa, o Flow Table Controller deverá possuir uma especificação funcional capaz de:

1. receber os metadados dos pacotes;
2. utilizar o hash_index recebido para calcular o endereço da Flow Table;
3. solicitar a leitura da entrada por meio do Data Mover;
4. identificar situações de MISS, HIT ou COLLISION;
5. criar novas entradas em caso de MISS;
6. atualizar as estatísticas de fluxos existentes em caso de HIT;
7. preservar entradas existentes em caso de colisão;
8. controlar a recepção de novos pacotes por meio de valid/ready;
9. aguardar a conclusão das operações de memória;
10. iniciar a finalização após o término do PCAP;
11. percorrer as entradas da Flow Table;
12. encaminhar os registros válidos ao Flow Finalizer;
13. sinalizar a conclusão global por meio de all_flows_done.

Os entregáveis desta etapa ficam organizados da seguinte forma:

Entregável	Resultado
1. Fluxograma	Representação do fluxo completo de MISS, HIT, COLLISION e finalização.
2. FSM	Estados, condições de transição e operações do Controller.
3. Sequência das operações	Descrição passo a passo do processamento dos pacotes e da finalização.
4. Dependências	Interfaces e decisões que precisam ser alinhadas com os demais módulos.

Esta especificação representa a primeira versão funcional do Controller. As interfaces, as latências e os detalhes de integração devem ser ajustados conforme a implementação efetiva do Data Mover, da Flow Table e do Flow Finalizer.

3. Estrutura da Flow Table

3.1 Organização da memória

A Flow Table armazena as estatísticas dos fluxos identificados durante o processamento dos pacotes. A organização definida considera 65.536 entradas lógicas, selecionadas por um índice de 16 bits.

Cada entrada lógica ocupa 512 bits, organizados em 32 words de 16 bits. Essa organização permite adaptar o registro lógico à largura física de memória utilizada.

A capacidade total é:

$$65.536 \times 512 \text{ bits} = 33.554.432 \text{ bits}$$

Isso corresponde a aproximadamente 4 MiB de memória.

3.2 Organização dos campos

A estrutura lógica de cada entrada é distribuída da seguinte maneira:

Words	Campo	Largura	Função
W0	valid + padding	16 bits	Indica se a posição contém um fluxo válido.
W1–W7	flow_key	104 bits	Identifica o fluxo armazenado.
W8–W9	packet_count	32 bits	Quantidade de pacotes registrados no fluxo.
W10–W13	byte_count	64 bits	Quantidade total de bytes registrados no fluxo.
W14–W18	first_timestamp	80 bits	Timestamp do primeiro pacote do fluxo.
W19–W23	last_timestamp	80 bits	Timestamp do pacote mais recente.
W24–W31	Reservado	128 bits	Espaço reservado para possíveis extensões.
Total		512 bits	Registro lógico completo.

Os campos são armazenados em words consecutivos de 16 bits. A ordem dos bits dentro de cada campo e a convenção de endereçamento dos words devem ser mantidas iguais entre o Controller, o Data Mover e a lógica de memória.

Os words W24–W31 permanecem reservados nesta versão. Eles não fazem parte das operações atuais de criação e atualização.

3.3 Endereçamento das entradas

O índice de 16 bits seleciona uma das 65.536 entradas lógicas.

Como cada entrada ocupa 32 words, o endereço-base é calculado por:

$$\text{base_address} = \text{hash_index} \times 32$$

Ou, em lógica binária:

$$\text{base_address} = \{\text{hash_index}, 5'b00000\}$$

O endereço de cada word é obtido adicionando o deslocamento correspondente ao endereço-base.

Por exemplo, para consultar os dados iniciais da entrada, o Controller utiliza os words W0–W7. Para atualizar as estatísticas de um fluxo existente, utiliza os words que contêm os contadores e o timestamp final.

3.4 Acesso à Flow Table

O Controller utiliza o Data Mover para acessar a memória. As operações principais são:

- Lookup: leitura de W0–W7 para verificar a validade da entrada e comparar a chave do fluxo.
- MISS: escrita dos campos de identificação e das estatísticas iniciais.
- HIT: leitura dos campos estatísticos e escrita dos valores atualizados.
- Finalização: leitura dos campos necessários de cada entrada válida durante a varredura da tabela.

A leitura e a escrita são controladas por solicitações e confirmações. O Controller deve aguardar `rd_valid` para utilizar os dados lidos e `wr_done` para considerar uma escrita concluída.

3.5 Chave do fluxo

O campo `flow_key` possui 104 bits e é formado pelos seguintes elementos:

Campo	Largura
<code>src_ip</code>	32 bits
<code>dst_ip</code>	32 bits
<code>src_port</code>	16 bits
<code>dst_port</code>	16 bits
<code>protocol</code>	8 bits
Total	104 bits

A chave é utilizada para verificar se a posição selecionada pelo hash realmente corresponde ao fluxo recebido. O hash determina a posição candidata, mas a comparação da chave é necessária para diferenciar um HIT de uma colisão.

Como o projeto considera uniflow inicialmente, a chave mantém a direção do tráfego. Assim, a tupla com origem e destino invertidos representa outro fluxo.

3.6 Exemplo de atualização de uma entrada

Considere a chegada de um pacote com os seguintes metadados:

- `flow_key`: chave de 104 bits;
- `packet_length`: 1200 bytes;

- timestamp: 5000;
- hash_index: 12.

Se a posição 12 estiver livre, o Controller cria uma nova entrada:

valid = 1

flow_key = chave recebida

packet_count = 1

byte_count = 1200

first_timestamp = 5000

last_timestamp = 5000

Se posteriormente chegar outro pacote do mesmo fluxo, com tamanho de 800 bytes e timestamp 5300, a entrada será atualizada para:

packet_count = 2

byte_count = 2000

first_timestamp = 5000

last_timestamp = 5300

O first_timestamp permanece inalterado após a criação do fluxo. O last_timestamp é atualizado a cada pacote aceito que pertence ao fluxo.

3.7 Cálculo da duração

O campo duration não é armazenado diretamente na Flow Table. Ele é calculado durante a finalização do fluxo, utilizando os timestamps registrados:

$duration = last_timestamp - first_timestamp$

Dessa forma, a Flow Table mantém o estado necessário para a extração incremental, enquanto o Flow Finalizer calcula as features derivadas ao final do processamento.

Observação importante: as larguras dos contadores e dos timestamps na estrutura física devem corresponder às larguras utilizadas nas interfaces e no Finalizer. Nesta especificação, packet_count utiliza 32 bits, byte_count 64 bits e cada timestamp 80 bits. Isso substitui a estrutura preliminar anterior de 297 bits por entrada, que utilizava contadores e timestamps menores e não corresponde à organização atual de 512 bits.